

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	T Jyothi Reddy	Reg. No.:	192472079
Section / Batch:	2024	Date:	

Code of Conduct

I T Jyothi Reddy (192472079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T Jyothi Reddy

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} \mid \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.
5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct

Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	4	4	4	4	4	20	92
2	4	4	4	4	4	20	
3	5	5	5	5	5	25	

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

1. MLE of Bigram Probability $P(\text{learning} \mid \text{machine})$

The Maximum Likelihood Estimate (MLE) for a bigram is:

$$P(w_2 \mid w_1) = C(w_1, w_2) / C(w_1)$$

Given:

$$C(\text{machine}, \text{learning}) = 3$$

$$C(\text{machine}) = 3$$

Therefore,

$$P(\text{learning} \mid \text{machine}) = 3 / 3 = 1$$

Therefore, the MLE is:

$$P(\text{learning} \mid \text{machine}) = 1 \text{ or } 100\%$$

This means that every occurrence of the word "machine" in the training corpus is followed by "learning". Hence, when a customer enters "machine", the e-commerce search system can strongly predict "learning" as the next word. This improves autocomplete and helps customers quickly find relevant search queries.

2. Backoff Model for "machine learning transforms"

The trigram probability is:

$$P(\text{transforms} \mid \text{machine}, \text{learning})$$

The sequence "machine learning transforms" does not occur in the corpus.

Therefore:

$$P(\text{transforms} \mid \text{machine}, \text{learning}) = 0$$

The model then backs off to the bigram:

$$P(\text{transforms} \mid \text{learning})$$

The sequence "learning transforms" is also not present.

Therefore:

$$P(\text{transforms} \mid \text{learning}) = 0$$

Finally, the model checks the unigram probability:

$$P(\text{transforms})$$

The word "transforms" does not occur in the corpus.

Therefore:

$$P(\text{transforms}) = 0$$

Hence, for this particular corpus:

$$P(\text{transforms}) = 0$$

Backoff is useful because when a trigram is unavailable, the model can use a bigram, and if the bigram is also unavailable, it can use a unigram. This allows the search system to handle previously unseen queries better. In this particular example, all three levels are unseen, so smoothing would be required to give "transforms" a non-zero probability.

3. Deleted Interpolation Probability of "machine learning improves"

The interpolation formula is:

$$P = \lambda_1 P(\text{trigram}) + \lambda_2 P(\text{bigram}) + \lambda_3 P(\text{unigram})$$

Given:

$$\lambda_1 = 0.5$$

$$\lambda_2 = 0.3$$

$$\lambda_3 = 0.2$$

Trigram Probability

$$P(\text{improves} \mid \text{machine}, \text{learning})$$

$$= C(\text{machine}, \text{learning}, \text{improves}) / C(\text{machine}, \text{learning})$$

$$= 1 / 3$$

$$= 0.3333$$

Bigram Probability

$$P(\text{improves} \mid \text{learning})$$

$$= C(\text{learning}, \text{improves}) / C(\text{learning})$$

$$= 1 / 3$$

$$= 0.3333$$

Unigram Probability

The total number of words in the corpus is 15.

The word "improves" occurs once.

Therefore:

$$P(\text{improves}) = 1 / 15$$

$$= 0.0667$$

Using the given interpolation weights:

$$P = (0.5 \times 0.3333) + (0.3 \times 0.3333) + (0.2 \times 0.0667)$$

$$P = 0.1667 + 0.1000 + 0.0133$$

$$P = 0.2800$$

Therefore:

Deleted Interpolation Probability ≈ 0.28

Interpolation improves robustness because it combines trigram, bigram and unigram information. Even when higher-order N-grams are sparse, lower-order information can still contribute to the final probability.

4. Entropy of the Prediction Distribution

Given:

$P(\text{improves}) = 0.33$

$P(\text{enables}) = 0.33$

$P(\text{drives}) = 0.33$

Entropy is calculated using:

$H = -\sum P(x) \log_2 P(x)$

Therefore:

$H = -[0.33 \log_2(0.33) + 0.33 \log_2(0.33) + 0.33 \log_2(0.33)]$

$H \approx 1.58$ bits

Therefore:

Entropy ≈ 1.58 bits

The entropy is relatively high because all three possible next words have almost equal probabilities. The system is therefore uncertain about which word will occur next. If one word had a probability close to 1, the entropy would be close to 0, indicating a highly predictable next word.

5. Python Program

```
import nltk
import numpy as np
import math
from nltk.util import ngrams
from collections import Counter

# Run these once if required:
# nltk.download('punkt')

# Corpus
corpus = """
machine learning improves business
machine learning enables automation
machine learning drives innovation
"""

# Tokenization
words = nltk.word_tokenize(corpus.lower())

# Create N-grams
unigrams = Counter(words)
bigrams = Counter(ngrams(words, 2))
trigrams = Counter(ngrams(words, 3))

print("===== N-GRAM COUNTS =====")
print("Unigrams:", unigrams)
print("Bigrams:", bigrams)
print("Trigrams:", trigrams)

# 1. Bigram MLE

p_learning_machine = (
    bigrams[("machine", "learning")] /
    unigrams["machine"]
)

print("\n===== BIGRAM MLE =====")
print("C(machine) =", unigrams["machine"])
print("C(machine, learning) =", bigrams[("machine", "learning")])
```

```
print("P(learning | machine) =", p_learning_machine)
```

2. Backoff Model

```
word = "transforms"
```

```
tri_count = trigrams[("machine", "learning", word)]
```

```
print("\n===== BACKOFF MODEL =====")
print("Trigram count =", tri_count)
```

```
if tri_count > 0:
    probability = tri_count / bigrams[("machine", "learning")]
    print("Using Trigram")
elif bigrams[("learning", word)] > 0:
    probability = (
        bigrams[("learning", word)] /
        unigrams["learning"]
    )
    print("Using Bigram")
elif unigrams[word] > 0:
    probability = unigrams[word] / len(words)
    print("Using Unigram")
else:
    probability = 0
    print("No lower-order N-gram available")
```

```
print("P(transforms) =", probability)
```

```
# -----
# 3. Deleted Interpolation
# -----
```

```
p_tri = (
    trigrams[("machine", "learning", "improves")] /
    bigrams[("machine", "learning")]
)
```

```
p_bi = (
    bigrams[("learning", "improves")] /
    unigrams["learning"]
)
```

```
p_uni = unigrams["improves"] / len(words)
```

```
lambda1 = 0.5
lambda2 = 0.3
lambda3 = 0.2
```

```
p_interpolation = (
    lambda1 * p_tri +
    lambda2 * p_bi +
    lambda3 * p_uni
)
```

```
print("\n===== DELETED INTERPOLATION =====")
print("Trigram probability =", p_tri)
print("Bigram probability =", p_bi)
print("Unigram probability =", p_uni)
print("Lambda1 =", lambda1)
print("Lambda2 =", lambda2)
print("Lambda3 =", lambda3)
print("Final probability =", p_interpolation)
```



```

# -----
# 4. Entropy
# -----

probabilities = np.array([0.33, 0.33, 0.33])

entropy = -sum(
    p * math.log2(p)
    for p in probabilities
)

print("\n===== ENTROPY =====")
print("Prediction probabilities:", probabilities)
print("Entropy =", entropy, "bits")

# -----
# 5. Final Prediction
# -----

predictions = {
    "improves": 0.33,
    "enables": 0.33,
    "drives": 0.33
}

print("\n===== NEXT-WORD PREDICTION =====")

for word, probability in predictions.items():
    print(word, "->", probability)

```

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

1. Penn Treebank POS Tags

Sentence 1

"Book an appointment with the doctor."

Book/VB

an/DT

appointment/NN

with/IN

the/DT

doctor/NN

Complete POS-tagged sentence:

Book/VB an/DT appointment/NN with/IN the/DT doctor/NN

Here, "book" is tagged as **VB (Verb)** because it represents an action, meaning to schedule an appointment.

Sentence 2

"The book contains medical information."

The/DT

book/NN

contains/VBZ

medical/JJ

information/NN

Complete POS-tagged sentence:

The/DT book/NN contains/VBZ medical/JJ information/NN

Here, "book" is tagged as **NN (Noun)** because it refers to a book containing information.

Thus, the same word "book" receives different POS tags depending on its context.

2. HMM Probability Calculation

Given:

$$P(\text{book} \mid \text{VB}) = 0.7$$

$$P(\text{book} \mid \text{NN}) = 0.3$$

$$P(\text{Start} \rightarrow \text{VB}) = 0.6$$

$$P(\text{Start} \rightarrow \text{NN}) = 0.4$$

For VB:

$$P(\text{VB}, \text{book}) = P(\text{Start} \rightarrow \text{VB}) \times P(\text{book} \mid \text{VB})$$

$$= 0.6 \times 0.7$$

$$= 0.42$$

For NN:

$$P(\text{NN}, \text{book}) = P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} | \text{NN})$$

$$= 0.4 \times 0.3$$

$$= 0.12$$

Therefore:

Probability of VB = 0.42

Probability of NN = 0.12

Since:

$$0.42 > 0.12$$

the HMM selects:

book = VB

The probabilistic model favors VB because both the transition probability from the start state and the emission probability of "book" are higher for VB than NN.

3. Rule-Based POS Tagging vs Stochastic HMM Tagging

Feature	Rule-Based Tagging	Stochastic HMM Tagging
Method	Uses manually defined rules	Uses probabilities
Training	Usually does not require training	Requires a tagged corpus
Context	Uses grammatical rules	Uses transition and emission probabilities
Explainability	High	Moderate
Flexibility	Limited	Better for varied language
Main limitation	Rules can become complex	Requires sufficient training data

In a healthcare chatbot, rule-based tagging is simple, predictable and easy to understand. However, it may fail when users use unusual or ambiguous language.

HMM tagging uses statistical information and context, making it more flexible. However, it depends on the quality and size of the training corpus.

4. Importance of Penn Treebank POS Tags

Standardized POS tags provide a consistent representation of grammatical information.

For example:

Book/VB an/DT appointment/NN

can help the chatbot understand that the user is requesting an action related to an appointment.

POS tags support:

1. **Intent Detection** – Identifying an appointment-booking request.
2. **Entity Identification** – Identifying important words such as "doctor" and "appointment".
3. **Response Generation** – Understanding sentence structure to generate an appropriate response.

Therefore, POS tagging can improve the performance of downstream NLP tasks in healthcare chatbots.

5. Python Program

```
import nltk
```

```
# Run these once if required:
```

```
# nltk.download('punkt')
```

```
# nltk.download('averaged_perceptron_tagger')
```

```
sentences = [
```

```
    "Book an appointment with the doctor.",
```

```
    "The book contains medical information."
```

```
]
```

```
print("===== TOKENIZATION AND POS TAGGING =====")
```

```
for sentence in sentences:
```

```
    tokens = nltk.word_tokenize(sentence)
```

```
    print("\nSentence:", sentence)
```

```
    print("Tokens:", tokens)
```

```

tags = nltk.pos_tag(tokens)

print("POS Tags:")
for word, tag in tags:
    print(word + "/" + tag, end=" ")

print()

# -----
# HMM Calculation
# -----

p_book_vb = 0.7
p_book_nn = 0.3

p_start_vb = 0.6
p_start_nn = 0.4

vb_probability = p_start_vb * p_book_vb
nn_probability = p_start_nn * p_book_nn

print("\n===== HMM CALCULATION =====")

print("\nVB:")
print("P(Start -> VB) =", p_start_vb)
print("P(book | VB) =", p_book_vb)
print("Joint Probability =", vb_probability)

print("\nNN:")
print("P(Start -> NN) =", p_start_nn)
print("P(book | NN) =", p_book_nn)
print("Joint Probability =", nn_probability)

if vb_probability > nn_probability:
    print("\nMost probable tag for 'book' = VB")
else:
    print("\nMost probable tag for 'book' = NN")

```

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

1. Applying the Transformation Rule

Initial POS tags:

market/NN growth/NN drives/NNS investment/NN

Transformation rule:

Change NNS to VBZ if the preceding word is tagged as NN.

The word "drives" has the tag NNS.

Its preceding word "growth" has the tag NN.

Therefore, the rule is applied:

drives/NNS → drives/VBZ

Corrected POS-tag sequence:

market/NN growth/NN drives/VBZ investment/NN

The transformation is linguistically appropriate because "drives" functions as a verb in the sentence and describes the action performed by the subject "Market growth".

2. Analysis of the Tagging Error

The sentence is:

"Market growth drives investment."

The initial tag for "drives" is:

drives/NNS

This is incorrect because NNS represents a plural noun.

In the sentence, "Market growth" is the subject and "drives" represents the action performed by the subject. Therefore, "drives" is a third-person singular present-tense verb.

The correct tag is:

drives/VBZ

The surrounding context helps identify this because:

growth/NN → drives → investment/NN

The transformation rule recognizes that an NNS tag following an NN tag is likely to be a verb in this grammatical structure.

3. Relative Frequency of Each Word

Given:

- market = 500
- growth = 350
- drives = 180
- investment = 420

Total frequency:

$$500 + 350 + 180 + 420 = 1450$$

Relative frequency is:

$$P(\text{word}) = \text{Frequency of word} / \text{Total frequency}$$

Market

$$P(\text{market}) = 500 / 1450$$

$$= \mathbf{0.3448}$$

Growth

$$P(\text{growth}) = 350 / 1450$$

$$= \mathbf{0.2414}$$

Drives

$$P(\text{drives}) = 180 / 1450$$

$$= \mathbf{0.1241}$$

Investment

$$P(\text{investment}) = 420 / 1450$$

$$= \mathbf{0.2897}$$

Word	Frequency	Probability
market	500	0.3448
growth	350	0.2414
drives	180	0.1241
investment	420	0.2897

Corpus frequency information can support probabilistic POS tagging by indicating how frequently words occur. It can also be used in language modeling, word prediction and statistical NLP systems.

4. Entropy Before and After Transformation

Entropy is calculated using:

$$H = -\sum P(w) \log_2 P(w)$$

Using the probabilities:

$$P(\text{market}) = 0.3448$$

$$P(\text{growth}) = 0.2414$$

$$P(\text{drives}) = 0.1241$$

$$P(\text{investment}) = 0.2897$$

Therefore:

$$H = -[0.3448 \log_2(0.3448) + 0.2414 \log_2(0.2414) + 0.1241 \log_2(0.1241) + 0.2897 \log_2(0.2897)]$$

Therefore:

$$\mathbf{Entropy \approx 1.916 \text{ bits}}$$

The transformation changes only the POS tag:

drives/NNS → drives/VBZ

It does not change the word frequencies. Therefore, the word-frequency distribution remains the same.

Hence:

$$\mathbf{Entropy \text{ before transformation} = 1.916 \text{ bits}}$$

$$\mathbf{Entropy \text{ after transformation} = 1.916 \text{ bits}}$$

$$\mathbf{Change \text{ in entropy} = 0}$$

This shows that the transformation improves POS-tagging accuracy but does not change the uncertainty of the word-frequency distribution.

5. Python Program

```
# -----  
# Corrected POS Tags  
# -----  
  
print("\n===== CORRECTED POS TAGS =====")
```

```

for word, tag in corrected_tags:
    print(word + "/" + tag)

# -----
# Word Frequency
# -----

frequency = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total = sum(frequency.values())

print("\n===== WORD FREQUENCIES =====")
print("Total frequency =", total)

probabilities = {}

for word, count in frequency.items():

    probability = count / total
    probabilities[word] = probability

    print(
        word,
        "Count =", count,
        "Probability =", round(probability, 4)
    )

# -----
# Entropy
# -----

entropy = 0

for probability in probabilities.values():
    entropy -= probability * math.log2(probability)

print("\n===== ENTROPY =====")
print("Entropy before transformation =",
      round(entropy, 4), "bits")

# Word frequencies do not change after transformation
entropy_after = entropy

print("Entropy after transformation =",
      round(entropy_after, 4), "bits")

print("Change in entropy =",
      round(entropy_after - entropy, 4), "bits")

# -----
# Final Result
# -----

print("\n===== FINAL POS TAGGING =====")

for word, tag in corrected_tags:
    print(word + "/" + tag)

```

Expected Final Output

===== INITIAL POS TAGS =====

market/NN
growth/NN
drives/NNS
investment/NN

===== TRANSFORMATION =====

drives/NNS -> drives/VBZ

===== CORRECTED POS TAGS =====

market/NN
growth/NN
drives/VBZ
investment/NN

===== WORD FREQUENCIES =====

Total frequency = 1450

market Count = 500 Probability = 0.3448

growth Count = 350 Probability = 0.2414

drives Count = 180 Probability = 0.1241

investment Count = 420 Probability = 0.2897

===== ENTROPY =====

Entropy before transformation = 1.9161 bits

Entropy after transformation = 1.9161 bits

Change in entropy = 0.0 bits

===== FINAL POS TAGGING =====

market/NN
growth/NN
drives/VBZ
investment/NN